

Overall Structure of Course

## Machine Learning

## Supervised

We have actual values for our predictions (goal)

One feature

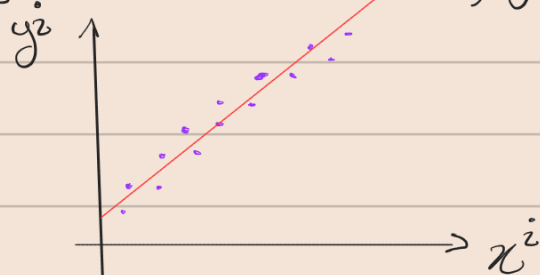
many features

## Unsupervised

we are classifying the given data set.

③ One feature data set with  $m$  training data.

imagine we have,  $x^i, y^i$  ;  $i=1, 2, \dots, m$  training data



to this kind of data linear function is fitting for predictions.

let, linear function to be,

$$f_{w,b}(x^i) = wx^i + b = y^i$$

\* Our aim is to find correct  $w, b$  values.

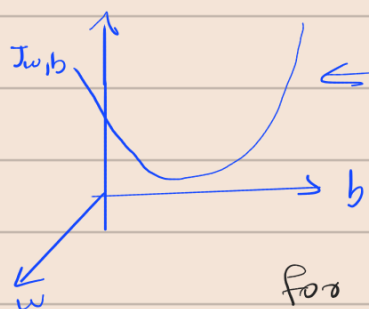
\* But there is always an error so, we create "Cost Function"

$$J_{w,b}(x^i) = \frac{1}{2m} \sum_{i=1}^m (\underbrace{\hat{y}^i - y^i}_{\text{difference of real and output value}})^2$$

so,  $J_{w,b}(x^i) = \frac{1}{2m} \sum_{i=1}^m (wx^i + b - y^i)^2$

so, the Cost function has two variables " $w, b$ "

egs:



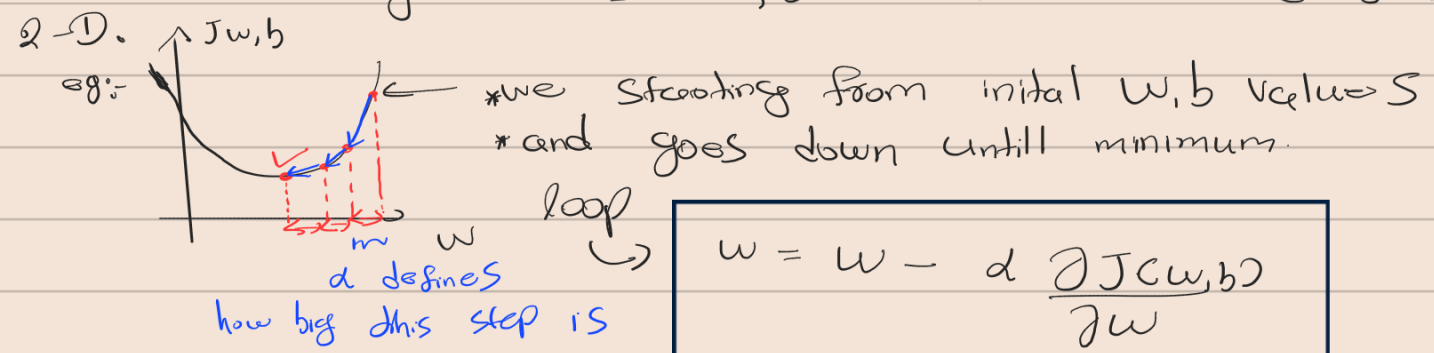
← This is a 3-D cone shape.

so, we want to find

$w, b$  s.t. " $J_{w,b}(x^i)$  is minimum."

for that we do "gradient descent"

When we looking variables  $w, b$  from each side function is



$$w = w - d \frac{\partial J(w, b)}{\partial w}$$

$$b = b - d \frac{\partial J(w, b)}{\partial b}$$

\* when  $w \approx w$  and  $b \approx b$  we stop and it's minimum of Cost.

\*  $d$  is learning rate

\*

$$J_{w, b} = \frac{1}{2m} \sum_{i=1}^m (wx^i + b - y^i)^2$$

$$\frac{\partial J_{w, b}}{\partial w} = \frac{1}{m} \sum_{i=1}^m (wx^i + b - y^i) x^i \quad \left\{ \begin{array}{l} \frac{\partial J_{w, b}}{\partial w} = \frac{1}{m} \sum_{i=1}^m (wx^i + b - y^i) x^i \\ \frac{\partial J_{w, b}}{\partial b} = \frac{1}{m} \sum_{i=1}^m (wx^i + b - y^i) \end{array} \right.$$

## Now what if we have "multi features"

in this case imagine we have  $j = 1, 2, \dots, n$  features.

So, linear regression will be,

$$f_{w,b}(x_j^i) = w_1 x_1^i + w_2 x_2^i + \dots + w_n x_n^i = \hat{y}^i$$

for easier we take  $w, x$  as vectors.

$$\vec{w} = [w_1 \ w_2 \ w_3 \ \dots \ w_n] \xrightarrow{\text{in matrix form}} \vec{w} = \begin{pmatrix} w_1 \\ w_2 \\ \vdots \\ w_n \end{pmatrix}$$

$b = b$  this is just scalar

$$\vec{x} = [x_1 \ x_2 \ x_3 \ \dots \ x_n]$$

multi linear regression

So,  $f_{\vec{w},b}(\vec{x}) = \vec{w} \cdot \vec{x} + b$  we use dot product here.

So, as above we do gradient descent

repeat {

$$J=1 \Rightarrow w_1 = w_1 - d \frac{1}{m} \sum_{i=1}^m (f_{\vec{w},b}(x^i) - y^i) x_1^i$$

$$J=2 \Rightarrow w_2 = w_2 - d \frac{1}{m} \sum_{i=1}^m (f_{\vec{w},b}(x^i) - y^i) x_2^i$$

$$J=n \Rightarrow w_n = w_n - d \frac{\partial J(\vec{w},b)}{\partial w}$$

$$b = b - d \frac{1}{m} \sum_{i=1}^m (f_{\vec{w},b}(x^i) - y^i)$$

\* in order to efficient method, training values  $x_j^i$  should be scale up or down.

Assume for one feature,  $\min < x_1 < \max$

1. dividing by max  $\Rightarrow x_1 = x_1 / \max$

2. mean normalization  $\Rightarrow x_1 = x_1 - \mu_1 / (\max - \min)$

3. Z-Score normalization  $\Rightarrow x_1 = \frac{x_1 - \mu_1}{\sigma_1}$

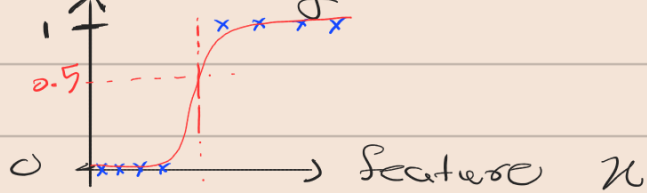
$\mu_1 \rightarrow$  mean of feature 1

$\sigma_1 \rightarrow$  standard deviation of feature 1

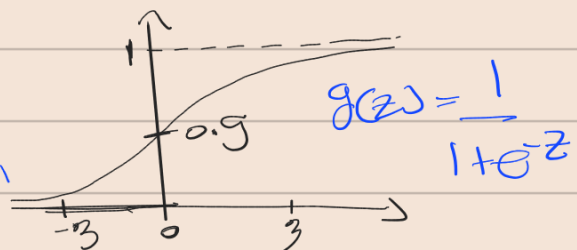
Now we move to unsupervised learning (Classification)

① Binary Classification.

we classify yes (1) or no (0)



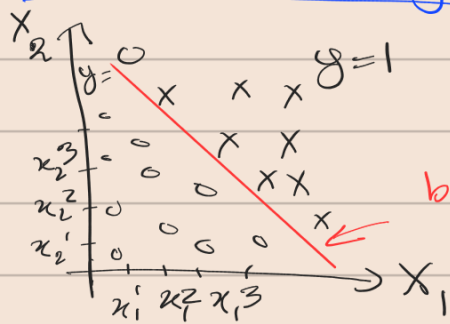
⇒ Sigmoid function



So, in logistic regression,

$$f_{\vec{w}, b}(\vec{x}) = g(z) = g(\underbrace{\vec{w} \cdot \vec{x} + b}_{z(\text{linear})}) = \frac{1}{1 + e^{-(\vec{w} \cdot \vec{x} + b)}}$$

Decision Boundary



boundary (our goal)

gives a value between 0 and 1, so it's like probability of happening  $P(y=1 | x; \vec{w}, b)$

$$y=1 \Rightarrow \frac{1}{1+e^{-z}} \geq 0.5$$

$$\Rightarrow z \geq 0$$

$$\Rightarrow \vec{w} \cdot \vec{x} + b \geq 0$$

$\therefore$  the boundary is,

$$\boxed{\vec{w} \cdot \vec{x} + b = 0}$$



$$f_{\vec{w}, b}(\vec{x}) = g(w_1 x_1 + w_2 x_2 + b)$$

let,

$$w_1 = w_2 = 1 \quad b = -3$$

$$\text{the boundary, } \vec{w} \cdot \vec{x} + b = 0$$

$$(w_1 + w_2)(x_1 + x_2) + b = 0$$

$$x_1 + x_2 = 3$$

Cost function for logistic regression

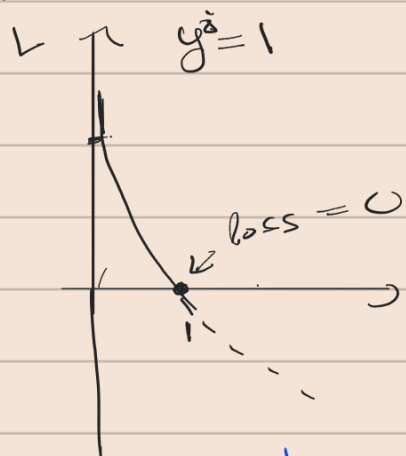
we introduce logistic loss function,

$$L(\underbrace{f_{\vec{w}, b}(\vec{x}^i)}_{\substack{\uparrow \\ \text{prediction} \\ (0.7, 0.3)}}, \underbrace{y^i}_{\substack{\uparrow \\ \text{real value} \\ (0, 1)}}) = \begin{cases} -\log(f_{\vec{w}, b}(\vec{x}^i)) & \text{if } y^i = 1 \\ -\log(1 - f_{\vec{w}, b}(\vec{x}^i)) & \text{if } y^i = 0 \end{cases}$$

so, Cost

$$J(\vec{w}, b) = \frac{1}{n} \sum_{i=1}^n L(f_{\vec{w}, b}(\vec{x}^i), y^i)$$

lets take same idea about LOSS function.



So make sense  $\rightarrow$  if  $f_{w,b}(x^i) = 0$

then loss is 0  
(wrong prediction)

### Simplified loss function

$$\text{earlier, } L(f_{w,b}(x^i), y^i) = \begin{cases} -\log(f_{w,b}(x^i)) & ; y^i = 1 \\ -\log(1 - f_{w,b}(x^i)) & ; y^i = 0 \end{cases}$$

simplify,

$$L(f_{w,b}(x^i), y^i) = \underbrace{-y^i \log(f_{w,b}(x^i))}_{\text{if } y^i = 0 \text{ this part cancel}} - \underbrace{(1-y^i) \log(1 - f_{w,b}(x^i))}_{y^i = 1 \text{ this part cancel}}$$

Now, Cost,

$$J(w,b) = \frac{1}{m} \sum_{i=1}^m [L(f_{w,b}(x^i), y^i)]$$

$$J(w,b) = \frac{1}{m} \sum_{i=1}^m [y^i \log(f_{w,b}(x^i)) + (1-y^i) \log(1 - f_{w,b}(x^i))]$$

### Gradient descent in logical regression

$$f_{w,b}(x) = \frac{1}{1 + e^{-(w \cdot x + b)}}$$

As earlier,

repeat  $\sum$

$$j=1,2,\dots,n \quad w_j = w_j - d \frac{\partial J(w,b)}{\partial w_j}$$

$$b = b - d \frac{\partial J(w,b)}{\partial b}$$

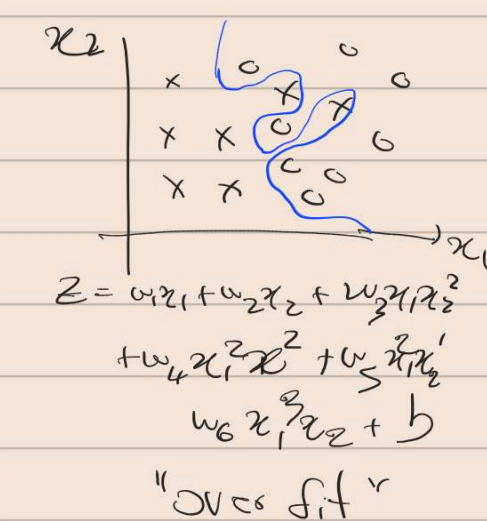
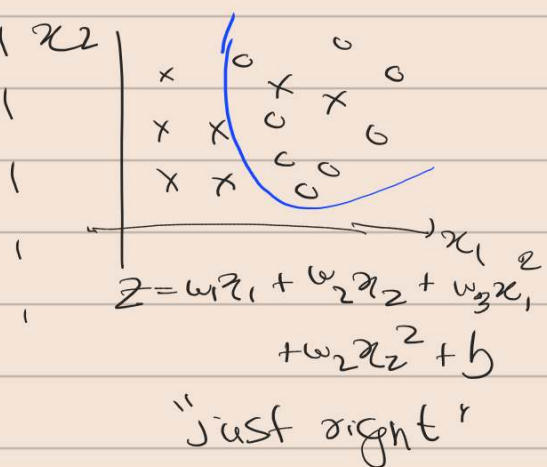
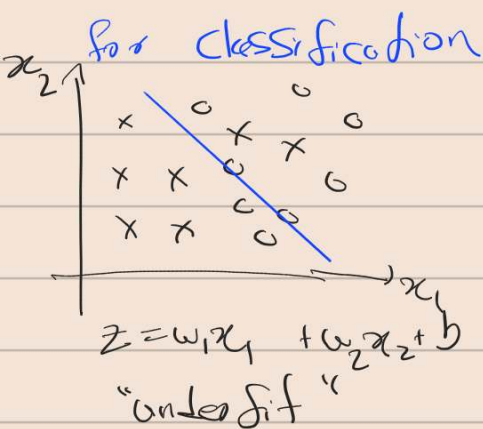
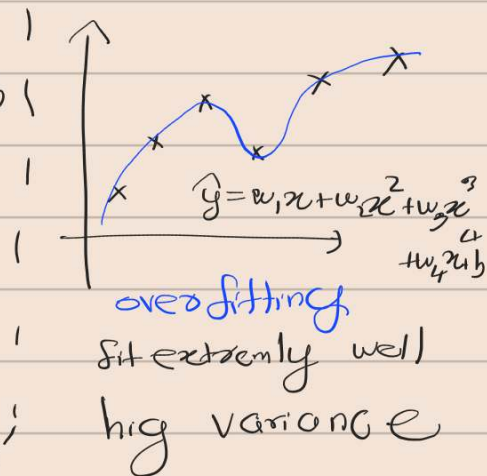
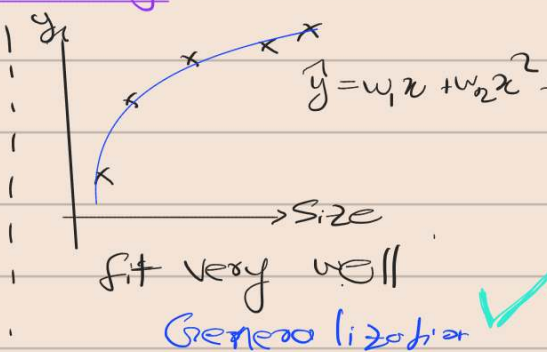
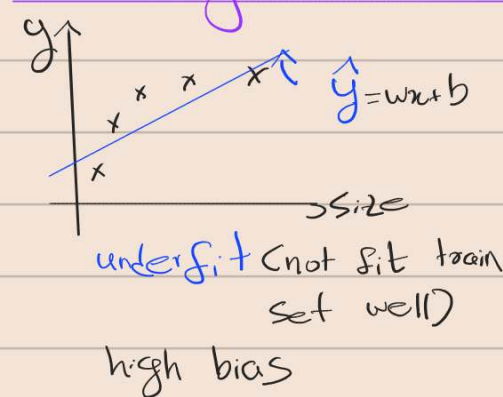
}

$$\frac{\partial J(w,b)}{\partial w_j} = \frac{1}{m} \sum_{i=1}^m (f_{w,b}(x^i) - y^i) x_j^i$$

\*Same as linear! but  $f_{w,b}$  is different.

$$\frac{\partial J(w,b)}{\partial b} = \frac{1}{m} \sum_{i=1}^m (f_{w,b}(x^i) - y^i)$$

# Overfitting and Underfitting



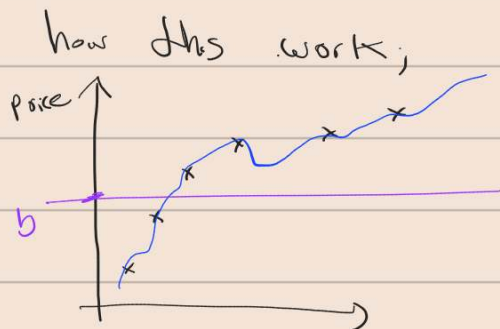
## for overfit

- ① increase number of training data
- ② feature selection (gather most valuable features only)
- ③ Regularization. (reduce parameters  $w_j = 0$  or  $w_j \approx 0$ )  
imagine  $j = 1, 2, \dots, n$  features,

we don't know what feature should regularize, (optional)

So,  $J(\vec{w}, b) = \frac{1}{2m} \sum_{i=1}^m (f(\vec{w}, b)(\vec{x}^i) - y^i)^2 + \underbrace{\frac{\lambda}{2m} \sum_{j=1}^n w_j^2}_{\text{regularization}} + \frac{\lambda}{2m} b^2$

$\lambda > 0$ ; regularization parameter



choose  $\lambda = 10^{10}$  (large)

$$f(\vec{w}, b)(\vec{x}) = w_1x + w_2x^2 + w_3x^3 + w_4x^4 + b$$

$\lambda = 0$  (overfit)

So, choose  $\lambda$

## Gradient descent with $\lambda$ :

repeat  $\{$

$$w_j = w_j - d \frac{\partial J(\vec{w}, b)}{\partial w_j} \rightarrow \frac{\partial J(\vec{w}, b)}{\partial w} = \left[ \frac{1}{m} \sum_{i=1}^m (f_{\vec{w}, b}(\vec{x}^i) - y^i) x_j^i + \frac{\lambda}{m} w_j \right]$$

$$b = b - d \frac{\partial J(\vec{w}, b)}{\partial b} \rightarrow \text{same as above}$$

$\}$  Simultaneous update

implementation:

$$w_j = w_j - d \underbrace{\frac{\lambda w_j}{m}}_{w_j (1 - d \frac{\lambda}{m})} - d \underbrace{\frac{1}{m} \sum_{i=1}^m (f_{\vec{w}, b}(\vec{x}^i) - y^i) x_j^i}_{\text{usual update}}$$

Shrink  $w_j$  if it small (eg:  $\lambda=1, d=0.01 \rightarrow (1 - 0.01 \times \frac{1}{90}) = 0.99998$  ↑ small)

For logistic regression

For logistic

$$J(\vec{w}, b) = -\frac{1}{m} \sum_{i=1}^m \left[ y^i \log(f_{\vec{w}, b}(\vec{x}^i)) + (1 - y^i) \log(1 - f_{\vec{w}, b}(\vec{x}^i)) \right] + \frac{\lambda}{2m} \sum_{j=1}^n w_j^2$$

gradient descent,

but it's not same as linear, it's with sigmoid function.

$$\text{repeat } \{ \quad w_j = w_j - d \frac{\partial J(\vec{w}, b)}{\partial w_j} \rightarrow \frac{\partial J(\vec{w}, b)}{\partial w} = \frac{1}{m} \sum_{i=1}^m \left[ (f_{\vec{w}, b}(\vec{x}^i) - y^i) x_j^i \right] + \frac{\lambda}{m} w_j$$

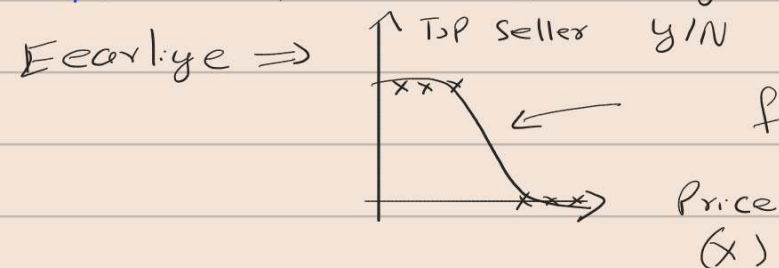
$$b = b - d \frac{\partial J(\vec{w}, b)}{\partial b} \rightarrow \text{same as above}$$

$\}$

# Idea of Neural Networks

eg:

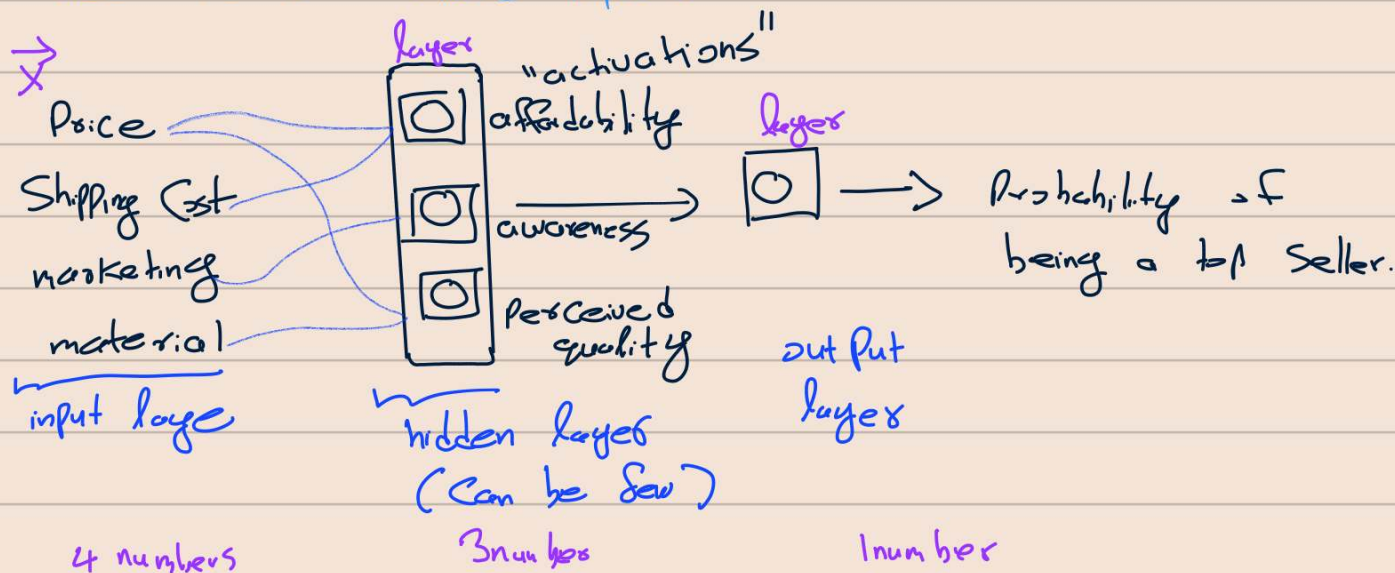
\* Demand Prediction (let say T-Shirt)



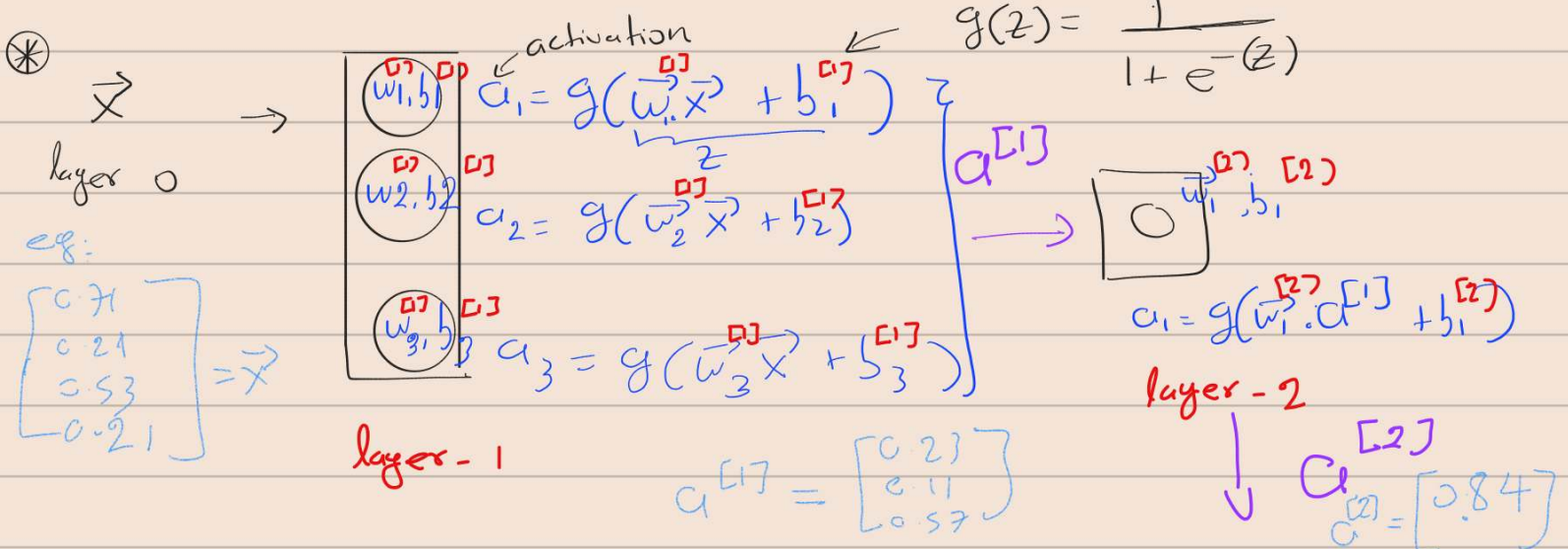
$$f(x) = \frac{1}{1 + e^{-(wx+b)}}$$

Sigmoid Function

here we add more inputs.



\*



so generally,

$$a_j^{[l]} = g(\vec{w}_j^{[l]} \cdot \vec{a}^{[l-1]} + b_j^{[l]})$$

l - layer  
j - neuron

"activation function"  $\rightarrow$  (gives activation of  $j^{th}$  neuron in  $l^{th}$  layer)

$\rightarrow$  in here sigmoid using but others there.

\* in library like tensorflow they do,

→ find  $w_1^1, b_1^1, w_2^2, b_2^2 \dots$  Calculate from above theorems.

→ get dot product and calculate  $z_1^1, z_2^1 \dots$

→ use Sigmoid (or other) and calculate  $a_1^1, a_2^1 \dots$

eg:-

def dense(a\_in, w, b):

units = w.shape[1] ← how many parameters in input

a\_out = np.zeros(units) ← [0, 0, 0]

for j in range(units):

w = w[:, j]

z = np.dot(w, a\_in) + b[j]

a\_out = g(z)

return a\_out. Sigmoid (or...)

then, sequential this as use activation values previous as a\_in, so last a\_out is the expected value.

but if use vectorize implementation,

def dense(a\_in, w, b)

z = np.matmul(a\_in, w) + b

return g(z)

import fun-1:-

$Z = (a_{in})^T W$  ← matrix multiplication so z can find easily

eg:-

$A \rightarrow \begin{bmatrix} 0 \\ 0 \\ 0 \\ 200 \\ 17 \end{bmatrix} \xrightarrow{A^T} \boxed{0} \rightarrow A^T$

↑  
feature values

so,  $A^T = \begin{bmatrix} 200 & 17 \end{bmatrix}$   
 $rel, w = \begin{bmatrix} 1 & -3 & 5 \\ -2 & 4 & -6 \end{bmatrix}$

So,  $z = A^T w + B$

## Model Training Steps.

Let's recall what we done in logistic regression.

①  $f_{\vec{w}, b}(\vec{x}) = \frac{1}{1 + e^{-z}}$  ;  $z = \vec{w} \cdot \vec{x} + b$

loss,

②  $L(f_{\vec{w}, b}(\vec{x}), y) = -y \ln(f_{\vec{w}, b}(\vec{x})) - (1-y) \ln(1-f)$

cost

③

$J(\vec{w}, b) = \frac{1}{n} \sum_{i=1}^m L(f_{\vec{w}, b}(\vec{x}^i), y^i)$

④ \* Train on data to minimize  $J(\vec{w}, b)$

$w = w - \alpha \frac{dJ(\vec{w}, b)}{dw}$        $b = b - \alpha \frac{dJ(\vec{w}, b)}{db}$

in python we do same,

① `model = Sequential([Dense(units=25, activation="sigmoid"), ...])` ← same as above

② `model.compile(loss = BinaryCrossentropy())`

or Mean Squared error

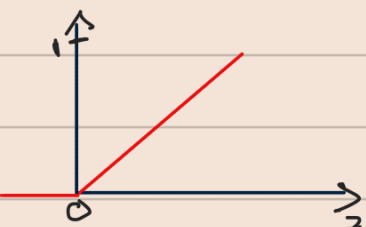
these are loss function in TensorFlow.

Keras.

③ `model.fit(X, y, epochs=100)`

↑  
100 iterations to update weight and minimize loss

## Other Activation Functions



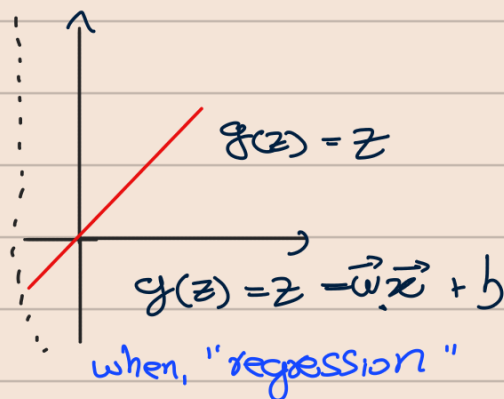
$f(z) = z \quad z > 0$

$f(z) = 0 \quad z < 0$

so,  $f(z) = \max(0, z)$

ReLU  
↑  
rectified

when "non-negative values" \* more common now



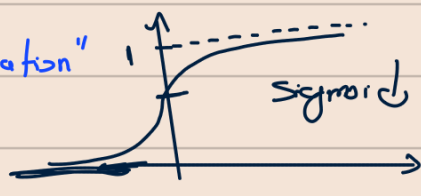
$f(z) = z$

$y(z) = z = \vec{w} \cdot \vec{x} + b$

when, "regression"

+ or - values

when, "binary-classification"



Sigmoid

$\frac{1}{1 + e^{-z}}$

← bit slow.

why  $\vec{z}$  activation function,  $\begin{bmatrix} 0 \\ \vdots \\ 0 \end{bmatrix} \rightarrow \vec{z}^{(2)} \dots$

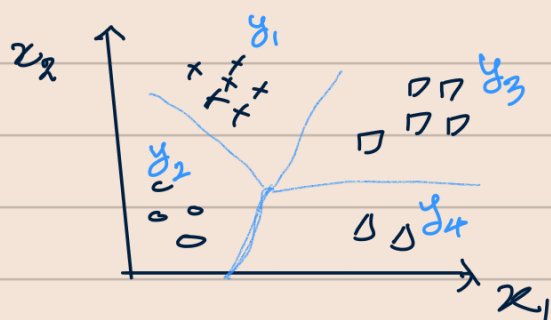
$$\vec{z}^4 = \underbrace{\vec{w}^4 \vec{a}^3}_{\text{again linear}} + b_1^4$$

again linear, if use ReLU

$$\vec{a}^4 = \frac{1}{1 + e^{-(\vec{w}^4 \vec{a}^3 + b_1^4)}}$$

again Sigmoid (but linear inside)

## Multi Class Classification



\* Softmax regression

in logistic regression (only 2 labels)

$$a_1 = g(z_1) = \frac{1}{1 + e^{-z_1}}$$

but now,

$$a_1 = \frac{e^{z_1}}{e^{z_1} + e^{z_2} + e^{z_3} + e^{z_4}}; z_1 = \vec{w}_1 \cdot \vec{x} + b_1$$

$$= P(y=1 | \vec{x})$$

like Bayes rule

$$= \frac{P(y=1 | \vec{x})}{\sum_{i=1}^4 P(y=i | \vec{x})}$$

$$= \frac{P(y=1 | \vec{x})}{P(y=1 | \vec{x}) + P(y=2 | \vec{x}) + \dots}$$

$$= \frac{e^{z_1}}{e^{z_1} + e^{z_2} + e^{z_3} + e^{z_4}}$$

$$a_2 = \frac{e^{z_2}}{e^{z_1} + e^{z_2} + e^{z_3} + e^{z_4}}; z_2 = \vec{w}_2 \cdot \vec{x} + b_2$$

if  $y_3 = y_4 = 0$  (So, logistic)

Same come. So this is "generalization" of logistic reg.

## Cost in Softmax regression.

we know,

$$a_1 = \frac{e^{z_1}}{\sum_{i=1}^n e^{z_i}} = P(y=1 | \vec{x})$$

$$a_n = \frac{e^{z_n}}{\sum_{i=1}^n e^{z_i}} = P(y=n | \vec{x})$$

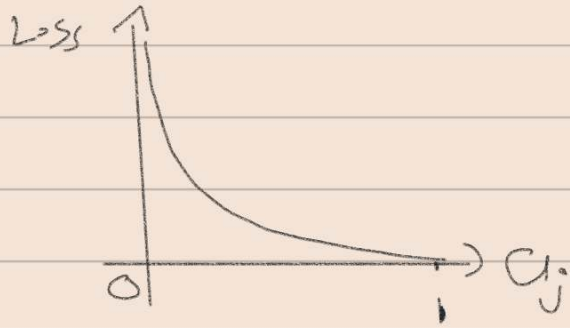
$$\text{loss}(a_1, \dots, a_n, y) = \begin{cases} -\log a_1, & y=1 \\ -\log a_2, & y=2 \\ \vdots \\ -\log a_n, & y=n \end{cases}$$

(in Tensor flow loss function for this)

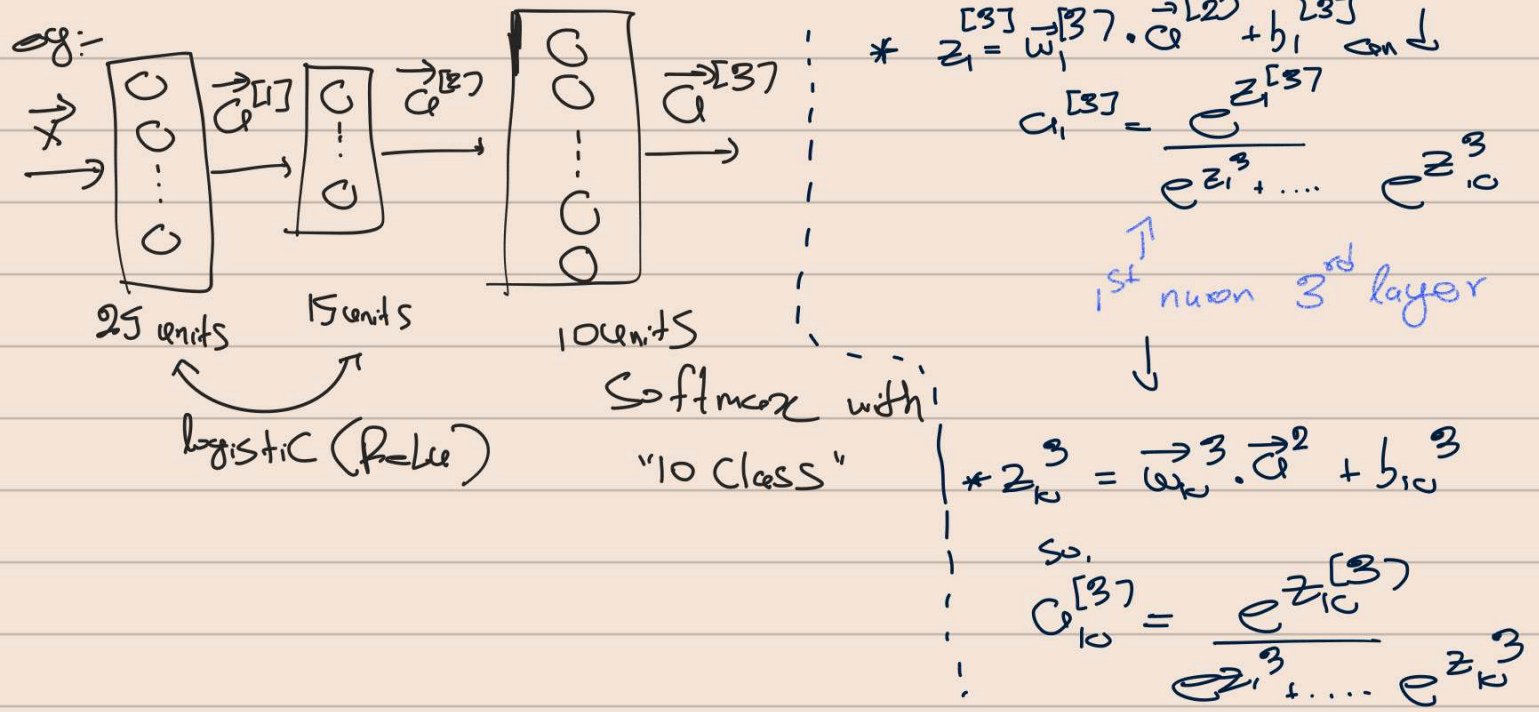
Sparse Categorical Crossentropy

So,  $y=j$

$$\text{loss} = -\log A_j : y=j$$



## Neural Network with Softmax output:



\* So, in Softmax,  $a_1, \dots, a_3$  cell activations are "function of all neurons"

## Numerical roundoff error

\*  $\frac{2}{1000} = (1 + \frac{1}{1000}) - (1 - \frac{1}{1000})$  but in Computers these two different.

So, in logistic regression for better accuracy

$$y = -y \log\left(\frac{1}{1+e^{-z}}\right) - (1-y) \log\left(1 - \frac{1}{1+e^{-z}}\right)$$

rather calculate  $e^z$  and substitute.

is more accurate

than

$$y = -y \log(a) - (1-y) \log(1-a) : a = \frac{1}{1+e^{-z}}$$

even both same.

So, in Python when calculating loss,

model.compile(loss=BinaryCrossEntropy (from logitS=True))

↑↑

for softmax loss,

model.compile (loss = sparse\_categorical\_crossentropy (from\_logits=True))  
 Same as above

$$L_{\vec{x}, y} = \left\{ -\log \left( \frac{e^{z_1}}{e^{z_1} + \dots + e^{z_{10}}} \right) \right.$$

not calculate "a" and substitute here.

multi class classification VS

the neurons in output layer  
 can get values more than (1,0.)

eg:

$$O^{[3]} = \begin{bmatrix} a_1^{[3]} \\ a_2^{[3]} \end{bmatrix}$$

1-10  
1-10  
Softmax

multi label classification

output layer has many  
 neurons

$$ex: O^{[3]} = \begin{bmatrix} a_1^{[3]} \\ a_2^{[3]} \\ a_3^{[3]} \end{bmatrix}$$

1 or 0  
1 or 0  
1 or 0  
↑ sigmoid activation

## Adam Algorithm intuition

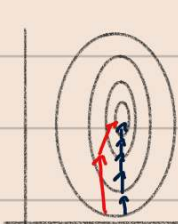
Adam = Adaptive moment estimation.

we know,

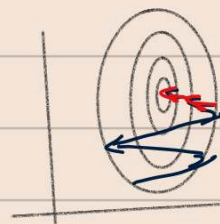
$$w_1 = w_1 - d_1 \frac{\partial J(\vec{w}, b)}{\partial w_1}$$

$$w_2 = w_2 - d_2 \frac{\partial J(\vec{w}, b)}{\partial w_2}$$

not just one fixed d.



if  $w_j$  moves some  
 direction  $d_j$  increase

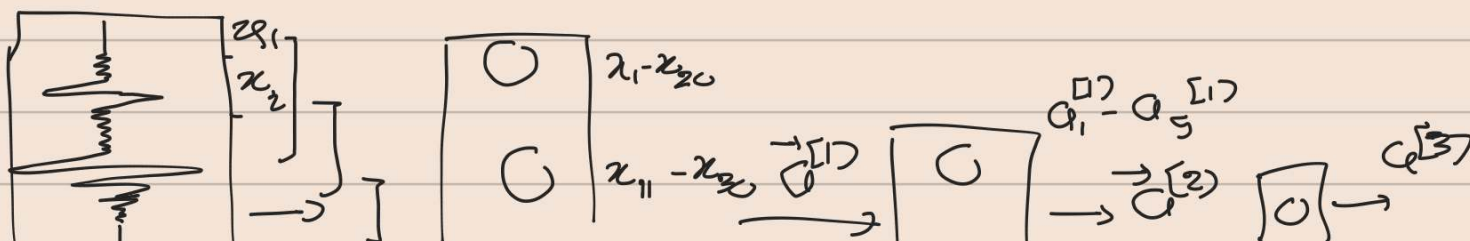


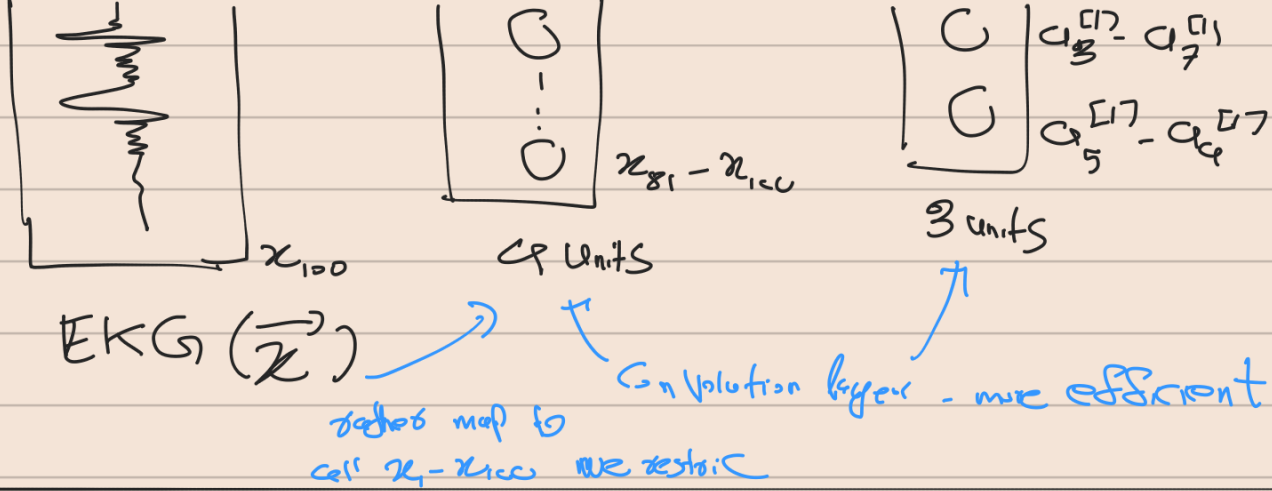
if  $w_j$  oscillating  
 $d_j$  reduce

So, in python,

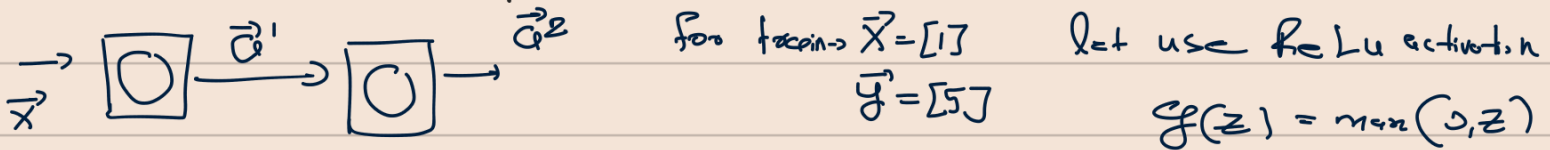
model.compile (optimizer = t.keras.optimizers.Adam (learning\_rate = 1e-3),

## Convolutional Neural Networks





Neural Network Example.



$$\vec{a}^1 = f(z) = f(w^1 x + b^1)$$

$$= w^1 x + b^1 = 2 \times 1 + 0 = 2$$

initial

$$\vec{a}^2 = w^2 x + b^2 = 3 \times 2 + 1 = 7$$

initial

$$J(w, b) = \frac{1}{2} (a^2 - y)^2 = 2$$

$$\frac{\partial J}{\partial a^2} = 2 \rightarrow \frac{\partial J}{\partial z^1} = 2 \dots$$

$$\frac{\partial J}{\partial w} = 6$$

update parameters ( $w_1, b_1, w_2, b_2 \dots$ )

To find  $\frac{\partial J}{\partial w}$  backward propagation

is a efficient method

## Applying ML

\* Evaluating model

| Data Set | Size                                                           | Price                                                          |                                              |
|----------|----------------------------------------------------------------|----------------------------------------------------------------|----------------------------------------------|
| 70%      | $\begin{cases} x^1 \\ x^2 \\ x^3 \\ \vdots \\ x^m \end{cases}$ | $\begin{cases} y^1 \\ y^2 \\ y^3 \\ \vdots \\ y^m \end{cases}$ | train set ( $x^{m_{train}}, y^{m_{train}}$ ) |
| 30%      | $\begin{cases} x^n \end{cases}$                                | $\begin{cases} y^n \end{cases}$                                | test set ( $x^{m_{test}}, y^{m_{test}}$ )    |

\* Fit parameters by minimizing cost function  $J(\vec{w}, b)$  ✓  
 $\vec{w}, b$

$$J(\vec{w}, b) = \left[ \frac{1}{2m_{\text{train}}} \sum_{i=1}^{m_{\text{train}}} (f_{\vec{w}, b}(\vec{x}_i^{\text{train}}) - y_i^{\text{train}})^2 + \frac{\lambda}{2m_{\text{train}}} \sum_{j=1}^n w_j^2 \right]$$

compute test Error:-

$$J_{\text{test}}(\vec{w}, b) = \frac{1}{2m_{\text{test}}} \left[ \sum_{i=1}^{m_{\text{test}}} (f(\vec{x}_i^{\text{test}}) - y_i^{\text{test}})^2 \right]$$

high

compute train Error.

$$J_{\text{train}}(\vec{w}, b) = \frac{1}{2m_{\text{train}}} \left[ \sum_{i=1}^{m_{\text{train}}} (f_{\vec{w}, b}(\vec{x}_i^{\text{train}}) - y_i^{\text{train}})^2 \right]$$

normally this is small

So, by compare  $J_{\text{test}}$  and  $J_{\text{train}}$  how well model working.  
 Specially on logistic regression as well.

## \* Model Selection

eg:-  $d=1$  1.  $f(\vec{x}) = w_1 x + b$

$d=2$  2.  $f(\vec{x}) = w_1 x + w_2 x^2 + b$

$d=10$  10  $f(\vec{x}) = w_1 x + w_2 x^2 + \dots + w_{10} x^{10} + b$

models

\* we look how  $J_{\text{test}}(\vec{w}, b)$  values are. ↓ lowest  
 \* if 5th order polynomial works well we choose that

to optimize this selection.

| X        | y        |                                                                                                                                                          |
|----------|----------|----------------------------------------------------------------------------------------------------------------------------------------------------------|
| $x_1$    | $y_1$    | 60% $(x^{m_{\text{train}}}, y^{m_{\text{train}}})$                                                                                                       |
| $x_2$    | $y_2$    |                                                                                                                                                          |
| $\vdots$ | $\vdots$ |                                                                                                                                                          |
| $x_m$    | $y_m$    | 20% $(x^{m_{\text{cv}}}, y^{m_{\text{cv}}}) \leftarrow$ <span style="border: 1px solid black; border-radius: 50%; padding: 2px;">Cross Validation</span> |
| $\vdots$ | $\vdots$ |                                                                                                                                                          |
| $x_n$    | $y_n$    | 20% $(x^{m_{\text{test}}}, y^{m_{\text{test}}})$ <span style="float: right;">or dev set</span>                                                           |

- \* Compute  $J_{\text{cv}}(\vec{w}, b)$  for different models (1st order, 2nd order, ...)
- \* Compute  $J_{\text{train}}(\vec{w}, b)$  and compare them for best model
- \* Use  $J_{\text{test}}(\vec{w}, b)$  to prove (independent from test, cv)

In Neural Networks:- Choose how many layers and units on each layer

## Bias and Variance

↑  
underfit

↑  
overfit

\* but if more variable in Model Can't plot them so,

\* High bias

just right

High Variance

$J_{train}$  ↑↑

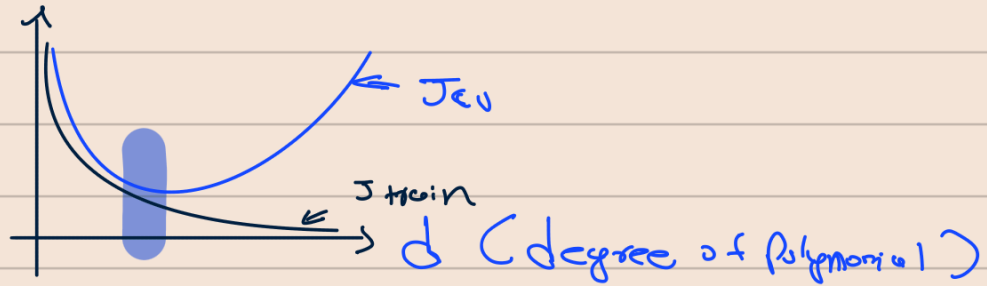
$J_{train}$  ↓↓

$J_{train}$  ↓↓

$J_{cv}$  ↑↑

$J_{cv}$  ↓↓

$J_{cv}$  ↑↑

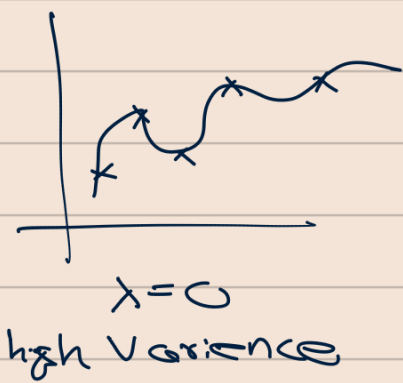
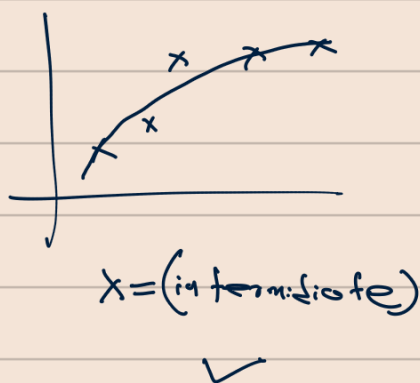
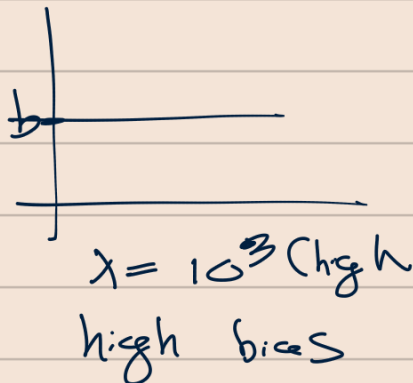


## Regularization and bias/Variance

for linear regression let's say,

$$f(\vec{x}) = w_1 x + w_2 x^2 + w_3 x^3 + w_4 x^4 + b$$

$$J_{\vec{w}, b} = \frac{1}{2m} \sum_{i=1}^m (f(\vec{x}^i) - y^i)^2 + \frac{\lambda}{2m} \sum_{j=1}^n w_j^2$$



Choose  $\lambda$  (regularize parameter)

try  $\lambda = 0 \rightarrow \min_{\vec{w}, b} J(\vec{w}, b) \rightarrow w', b'$   
 set  $\lambda$  train set error

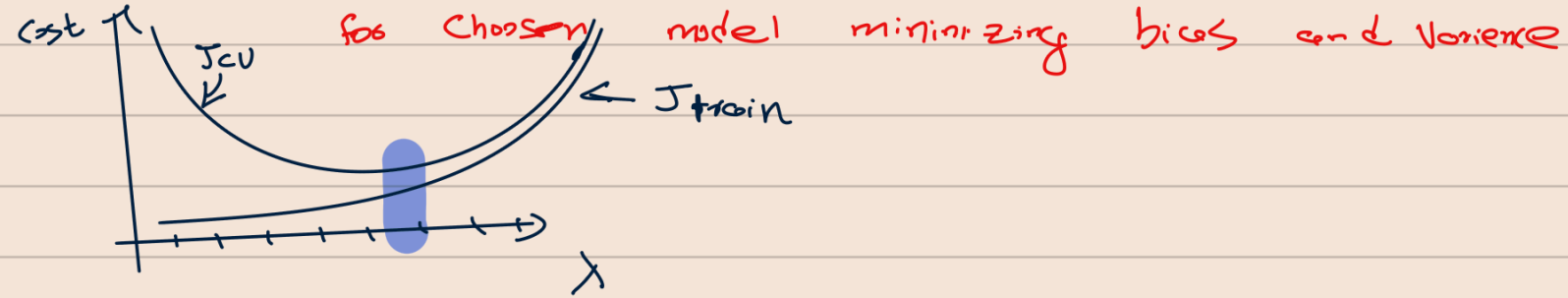
← "already Chosen model" the

$\rightarrow J_{cv}(w', b')$   
 Cross Validation error

$J_{cv}(w^5, b^5)$

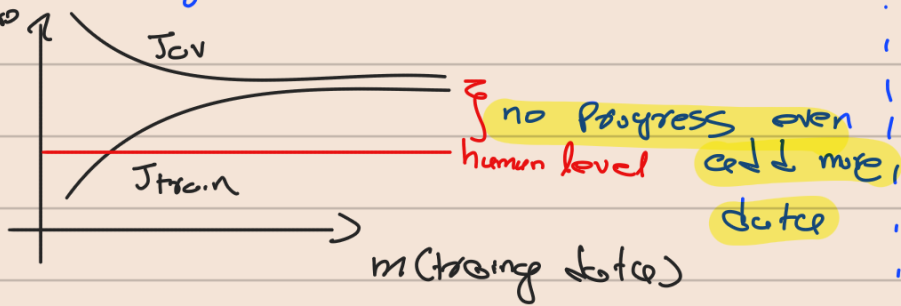
if this lowest choose that "X"

in TensorFlow  $\Rightarrow$  kernel\_regularizer = L2(0.01)



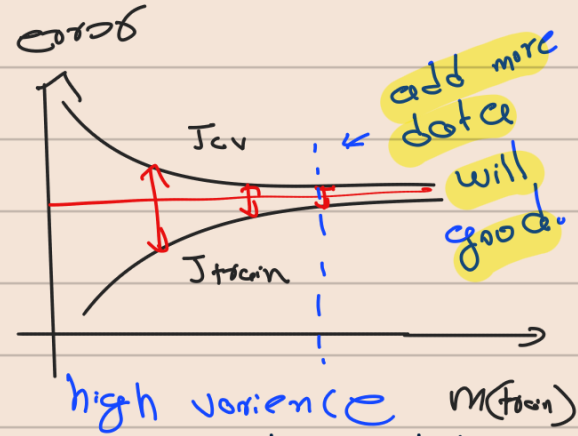
## \* Learning Curves.

Let say we have a model.



high bias

- \* Try additional sets features ( $x_1, x_2$ )
- \* Try adding Polynomial features
- \* decrease  $\lambda$

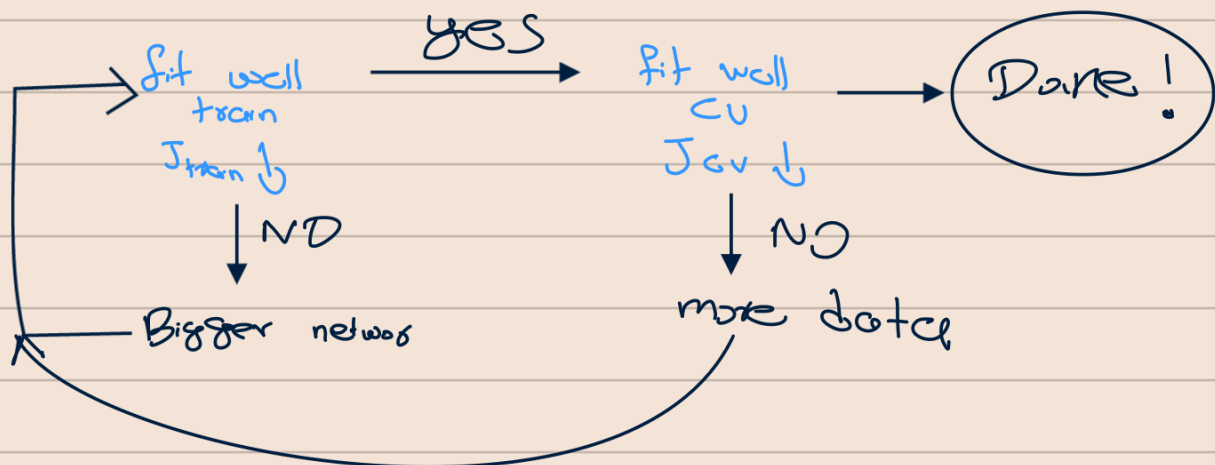


high variance

- \* get more train data
- \* Try smaller sets features
- \* increase  $\lambda$

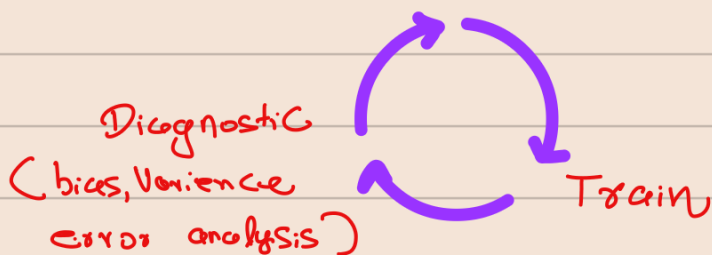
in neural networks

- \* Bigger network (more layers) : cpu power
- \* More data : Hard to find data



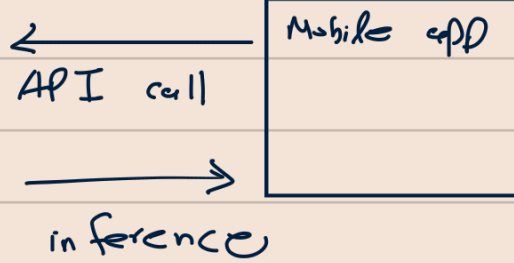
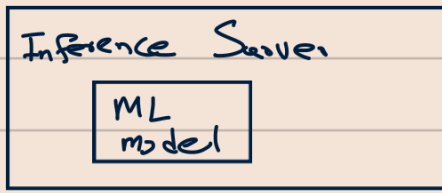
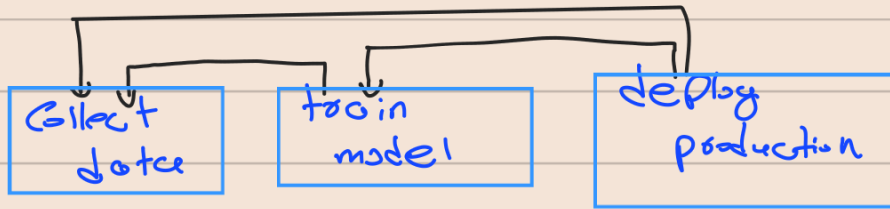
Iterative loop for ML development.

choose Architecture (Model, data, etc)



in a project

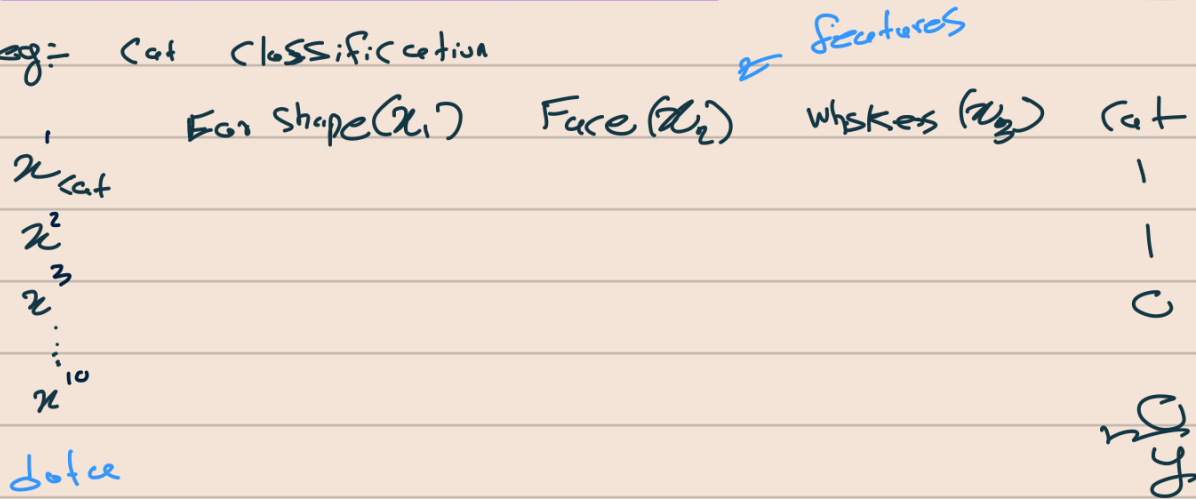
Scope of Project



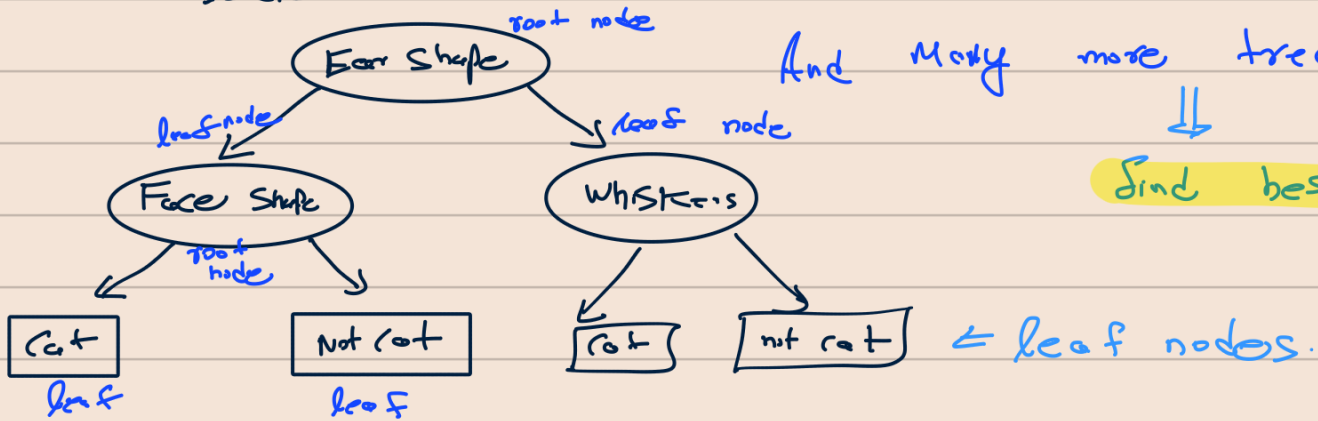
## Decision tree model

## Classification Trees

eg:- Cat classification



### Decision Tree.



1.) what feature split on each node.  
(depth)

2.) when stop splitting.

Entropy (measure of impurity.)

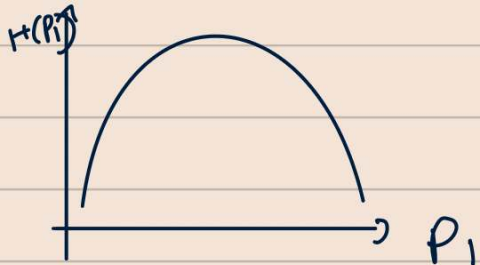
$$P_1 \rightarrow \frac{\text{cat}}{\text{total}}$$

$$P_1 + P_0 = 1$$

$$H(P_1) = -P_1 \log_2(P_1) - P_0 \log_2(P_0)$$

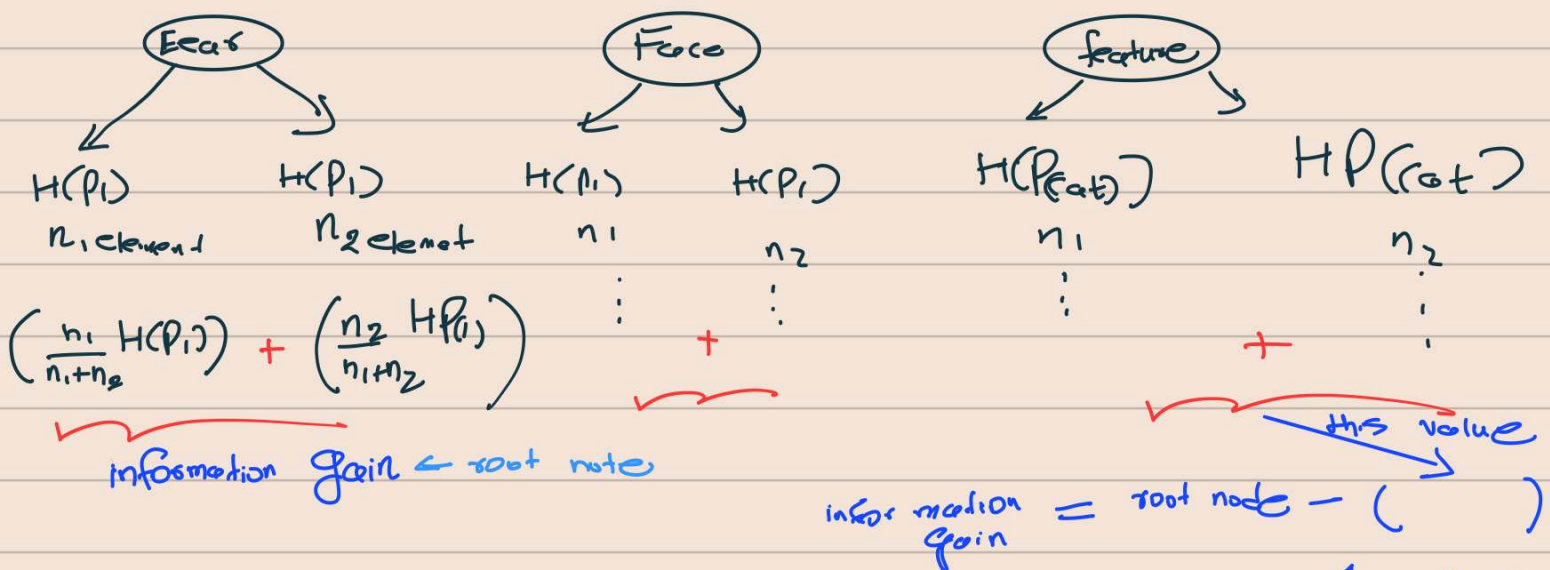
$$= -P_1 \log_2(P_1) - (1 - P_1) \log_2(1 - P_1)$$

entropy  
impurity of  $P_1$



Note:  $\log 0 = 1$

## ① What feature split



- \* Method
- ① Start with all possible root node  $\uparrow$  if leaf node
  - ② Pick highest information gain (put it on top)
  - ③ with that, create tree with left, right branches
  - ④ Stop at, node 100% one class
- exceeding maximum depth  
information gain < threshold.  
elements in a node < threshold

## More than 2 possible outcomes for a feature

### Method:

\* If a categorical feature can take  $K$  values, create  $K$  binary features (0 and 1) one hot encoding

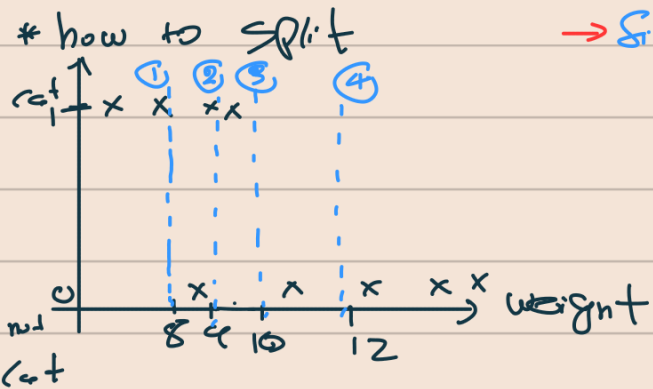
eg:  
Ears shape: Pointy, oval, floppy

| Pointy | floppy | Face | Whiskers |
|--------|--------|------|----------|
| 1      | 0      | 0    | 1        |
| 0      | 1      | 0    | 1        |
| 1      | 0      | 0    | 0        |
| 0      | 0      | 1    | 1        |

## Continuous Valued features

Eg: Add a feature like

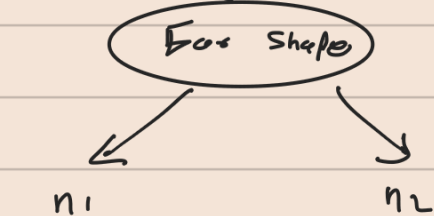
weight



→ find information gain for each 1, 2, 3, ...  
 → find maximum  
 ↑  
 this is the information gain for this feature.

## For Regression Trees

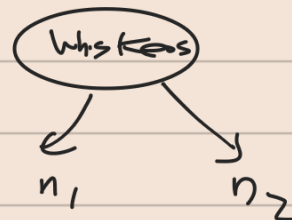
① Choosing a split



value  $V_1$   $V_2$



$V_3$   $V_4$



$V_5$   $V_6$

$$\left( \frac{n_1}{n_1+n_2} V_1 + \frac{n_2}{n_1+n_2} V_2 \right)$$

↓

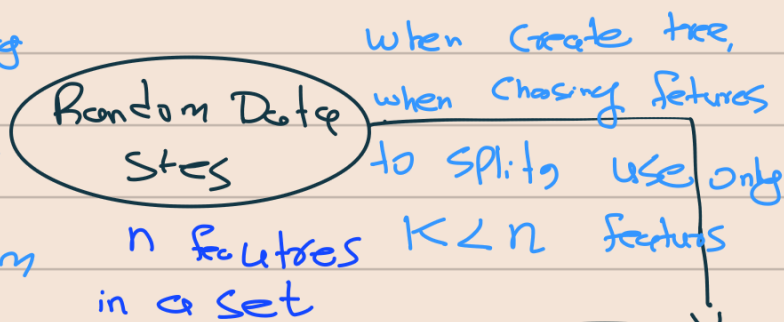
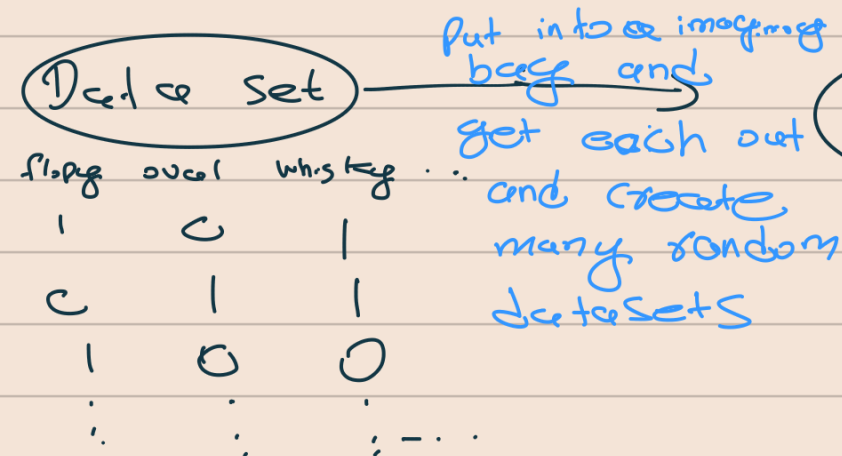
$$\left( V_{\text{root node}} \right)$$

$$\frac{n_1}{n_1+n_2} V_3 + \frac{n_2}{n_1+n_2} V_4$$

$$\left( \right) \quad \left( \right)$$

target.

## Creating more trees

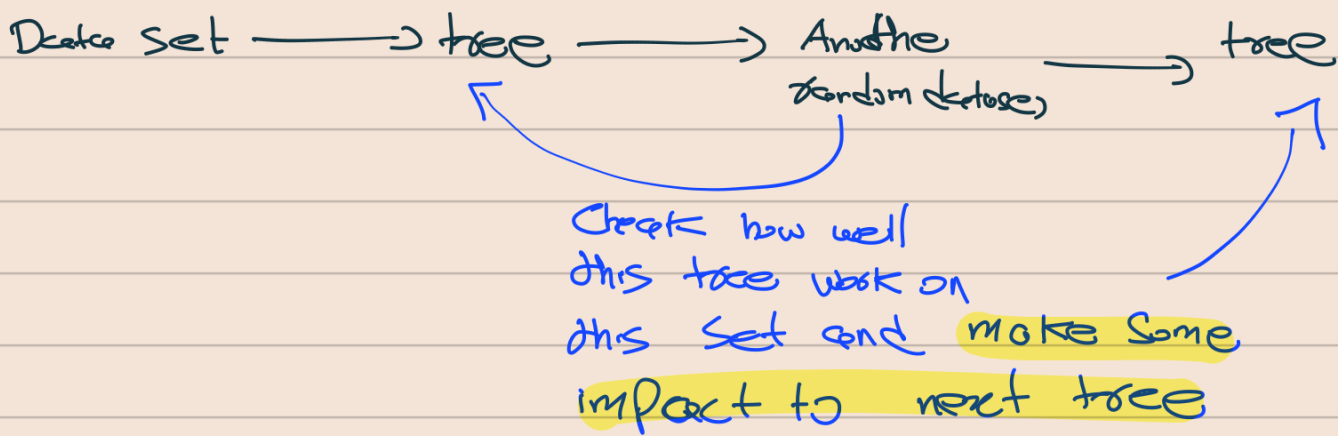


Random forest

Random forest tree

From them create different trees into above method

## XG Boost (extreme gradient boost)



## Unsupervised Learning Recommenders, Reinforce Learning

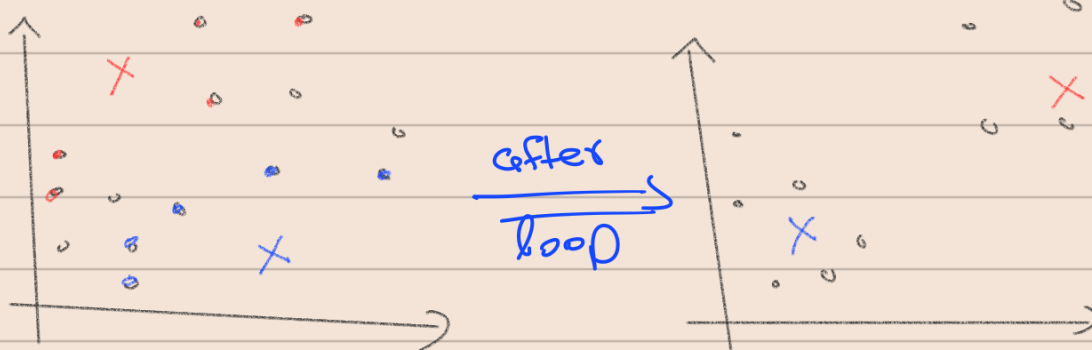


no targeting to look for something "interesting" (can group)

## K-means Algorithm

Step ①  $\Rightarrow$  Make initial guess for cluster centroids  
And Assign each point to its closest centroid

Step ②  $\Rightarrow$  Find coverage of each assigned values and Re compute the centroid.



## The Algorithm

Randomly initialize K cluster centroids  $\mu_1, \mu_2, \dots, \mu_K$

Repeat  $\{$  for  $z=1$  to  $m \leftarrow m$  training data

the index value 1 to K for each data  $\rightarrow C^z := \text{index (from 1 to K) of cluster centroid closest to } x^z$   
 $\min_k \|x^z - \mu_k\|^2$   
 Vector of n features

for  $K=1$  to  $K$

$\mu_K := \text{Avg (mean) of points assigned to cluster K}$

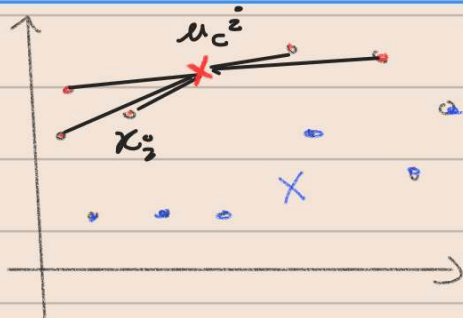
}

eg:  $\mu_1 = \frac{1}{4} [x^{(1)} + x^{(5)} + x^{(7)} + x^{(30)}]$   
 a vector of n features      vectors of n features

let  $\mu_{C^z}$  = cluster centroid of, the cluster assigned to  $x^z$   
 so, Cost function,

$$J(C^1, \dots, C^m, \mu_1, \dots, \mu_K) = \frac{1}{m} \sum_{z=1}^m \|x^z - \mu_{C^z}\|^2$$

distortion

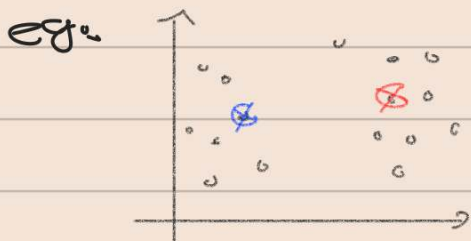


minimize  $K$ , tune  $\mu_1, \dots, \mu_K$

when to stop? Cost stays const  
 Cost reducing very slowly.

## Initialize

\* Choose  $K$  ( $K \leq m$ ) training examples.  
 \* set  $\mu_1, \mu_2, \dots, \mu_K$  to these  $K$  examples.



## Random initialization

for  $z=1$  to  $100 \}$

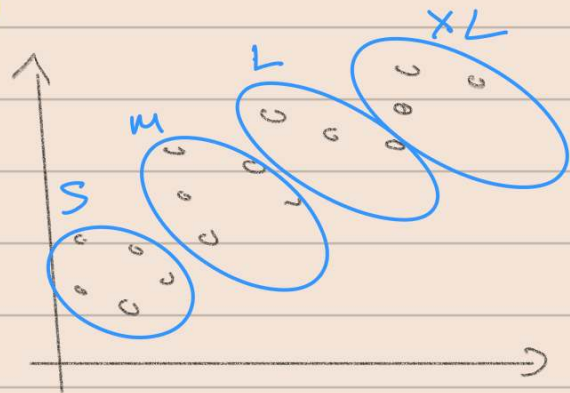
initialize  $K$

Run K-mean Algorithm and  
 Compute Cost ( $J$ )

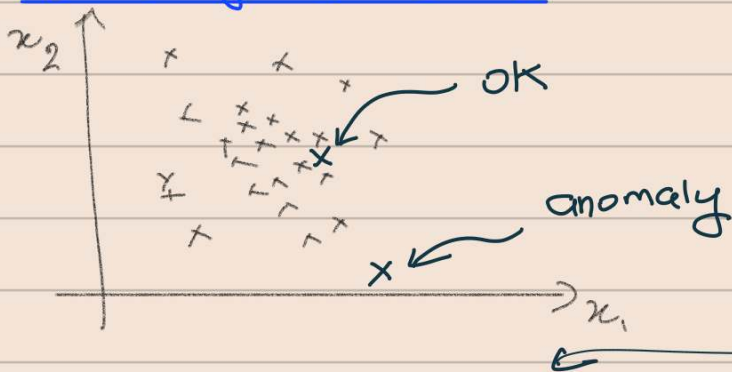
in this method

## Choosing KC number of clusters

Evaluate K-means based on how well it performs on that later purpose

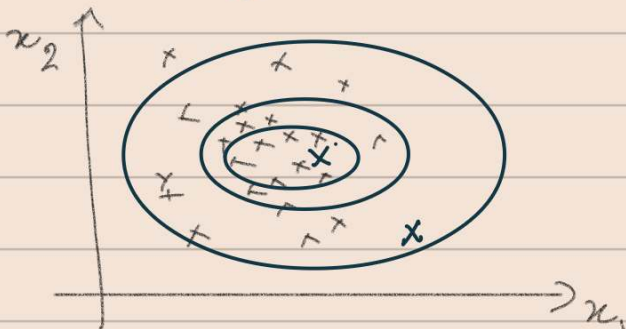


## Anomaly detection



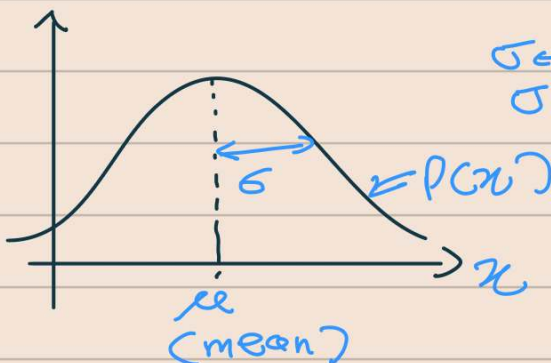
how to tell,  $x_{test}$  anomalous?

## Density estimation.



$P(x_{test}) < \epsilon$  or  $P(x_{test}) \geq \epsilon$   
anomaly OK

## Gaussian (Normal) distribution

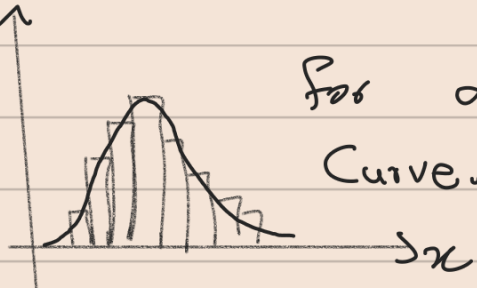


$\sigma$  = Standard deviation  
 $\sigma^2 \rightarrow$  Variance

$$P(x) = \frac{1}{\sqrt{2\pi}\sigma} e^{-\frac{(x-\mu)^2}{2\sigma^2}}$$

$$\sigma^2 = \frac{1}{N} \sum_{i=1}^N (x_i - \mu)^2$$

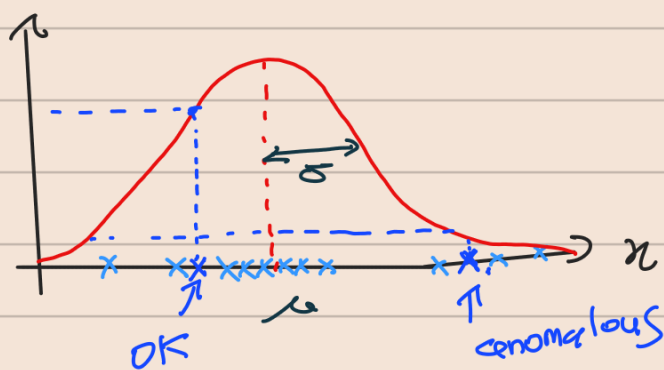
in reality:-



For  $\alpha$  bones, it's a complete bell Curve.

Parameter Estimation (One feature)

$$\text{DataSet} = \{x^1, x^2, \dots, x^3\}$$



$$\mu = \frac{1}{m} \sum_{i=1}^m x^i \quad \checkmark$$

$$\sigma^2 = \frac{1}{m} \sum_{i=1}^m (x^i - \mu)^2 \quad \checkmark$$

we now can find  $P(x)$  ✓

with  $P(x) \leq \epsilon$  can estimate.

n features

$$\vec{x} = \begin{pmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{pmatrix}$$

$$P(\vec{x}) = P(x_1, \mu_1, \sigma_1^2) * P(x_2, \mu_2, \sigma_2^2) * \dots$$

$$P(x_n, \mu_n, \sigma_n^2)$$

$$P(\vec{x}) = \prod_{i=1}^n P(x_i, \mu_i, \sigma_i^2)$$

anomaly detection Algorithm:

1) Choose n features  $x_i$  that might indicative of anomalous

2) Fit parameters

$$\left| \begin{aligned} \mu_j &= \frac{1}{m} \sum_{i=1}^m x_j^{(i)} \\ \text{or} \\ \text{if vectorize} \end{aligned} \right| \left| \begin{aligned} \sigma_j^2 &= \frac{1}{m} \sum_{i=1}^m (x_j^{(i)} - \mu_j)^2 \end{aligned} \right| \begin{aligned} j &\rightarrow \text{feature} \\ i &\rightarrow \text{train data} \\ & \quad (m \text{ data}) \end{aligned}$$

$$\vec{\mu} = \frac{1}{m} \sum_{i=1}^m \vec{x} = \begin{pmatrix} \mu_1 \\ \mu_2 \\ \mu_3 \\ \vdots \\ \mu_n \end{pmatrix}$$

3) for new example  $x$ , compute  $P(x)$

$$P(x) = \prod_{j=1}^n P(x_j, \mu_j, \sigma_j^2) = \prod_{j=1}^n \frac{1}{\sqrt{2\pi}\sigma_j} \exp\left(-\frac{(x_j - \mu_j)^2}{2\sigma_j^2}\right)$$

anomaly if  $P(x) < \epsilon$

How to Choose  $\epsilon$

eg: after years data,

$10^4 \rightarrow \text{normal } (y=0)$

$20 \rightarrow \text{normal } (y=1)$

Train: 6000 ( $y=0$ )

CV: 2000 ( $y=0$ )

10 ( $y=1$ )

tune  $\epsilon$  tune  $x_j$

test: 2000 ( $y=0$ ),

10 ( $y=1$ )

Algorithm

1) Fit model  $P(x)$  on training set  $(x^1, x^2, \dots, x^m)$

On a cross validation/test example  $x$ , predict

$$y = \begin{cases} 1 & ; P(x) < \epsilon \\ 0 & ; P(x) \geq \epsilon \end{cases}$$

2) use Cross Validation (CV) set to choose  $\epsilon$

|                                                                  |                                                                             |
|------------------------------------------------------------------|-----------------------------------------------------------------------------|
| future anomalies may look<br>nothing like before examples before | possible examples in future<br>likely to be similar to<br>ones in train set |
| $\Downarrow$<br>Anomaly Detection                                | $\Downarrow$<br>Supervised Learning                                         |
| eg:- Fraud Detection                                             | eg:- Email Spam Classifier                                                  |
| • Manufacturing (new ways defects)                               | • Manufacturing (always same defects)                                       |
| • Monitoring machines in data<br>centers.                        | • Diseases classifier                                                       |

## non - gaussian features

$$x_1 \xrightarrow{\text{try}} \log(x_1)$$

$$x_2 \xrightarrow{\text{try}} \log(x_2 + 1)$$

$$x_3 \xrightarrow{\text{try}} \sqrt{x_3}$$

$$x_4 \xrightarrow{\text{try}} x_4^{1/3}$$

## Error handle

\* Try add more (or make from existing ones) features

eg:- dcl no centre

$x_1$  - RAM  $x_2$  - temp

$x_3$  - CPU use  $x_4$  - network traffic

$$x_5 = \frac{\text{CPU use}}{\text{network traffic}} \Rightarrow \text{if } x_5^T \text{ anomaly}$$

$$x_6 = \frac{x_3^2}{x_4}$$

## Recommender Systems.

### \* Making Recommendations.

eg:-

| Movie                                                                                                                                                     | names of people |
|-----------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------|
| <div style="border: 1px solid black; border-radius: 50%; padding: 5px; display: inline-block;">                     movie<br/>name                 </div> |                 |

rate (0-5 or ?)

eg:-  $n_u = 4$   $n_m = 5$

$$r(1,1) = 1 \Rightarrow y(1,1) = 4$$

$$r(2,1) = 0$$

how much the rate

$n_u$  = no. of users

$n_m$  = no. of movies

$r(i,j)$  =  $i^{\text{th}}$  movie, rated by  $j^{\text{th}}$  user

booked or not

$y(i,j)$  = actual rating for  $i^{\text{th}}$  movie by  $j^{\text{th}}$  user

### \* when we add features of the movie.

| Movie | David | Bob | $x_1$<br>romance | $x_2$<br>action |
|-------|-------|-----|------------------|-----------------|
| 1     |       |     | 0.09             | 0               |
| 2     |       |     |                  |                 |
| 3     |       |     | 0.1              | 1.0             |

$3^{\text{rd}}$  movie eg:  $x^3 = \begin{bmatrix} 0.1 \\ 1.0 \end{bmatrix}$

$x_{12}$  feature

$$x^1 = \begin{bmatrix} 0.09 \\ 0 \end{bmatrix}$$

predicting action for  $i^{\text{th}}$  movie AS:-

$$w_i^j x^j + b^j$$

linear regression

Parameter (Matrix)

eg:- let  $w' = \begin{bmatrix} 5 \\ 0 \end{bmatrix}$  so,  $w'x' + b = \begin{bmatrix} 5 \\ 0 \end{bmatrix} \begin{bmatrix} 0.99 \\ 0 \end{bmatrix} + 0 = 4.95$

Parameter for user  $j=1$

Prediction for  $z=1$  movie by  $j=1$  user.

### Cost Function

$m^j =$  no. of movies rated by  $j^{\text{th}}$  user

$$J(w^j, b^j) = \frac{1}{2m^j} \sum_{z:r(z,j)=1} (w^{(j)} \cdot x^z + b^{(j)} - y^{(z,j)})^2 + \frac{\lambda}{2m^j} \sum_{k=1}^n (w_k^j)^2$$

min

$\frac{1}{2m^j}$  Cost

$\sum_{z:r(z,j)=1}$  only take rated movies

$\sum_{k=1}^n$  no. of features

$\frac{\lambda}{2m^j} \sum_{k=1}^n (w_k^j)^2$  regularization (for overfitting)

For all users and generalize one

$$J(w^1, \dots, w^{n_u}, b^1, \dots, b^{n_u}) = \frac{1}{2} \sum_{j=1}^{n_u} \sum_{z:r(z,j)=1} (w^j \cdot x^z + b^j - y^{(z,j)})^2 + \frac{\lambda}{2} \sum_{j=1}^{n_u} \sum_{k=1}^n (w_k^j)^2$$

Since  $m^j$  is cost

$\sum_{j=1}^{n_u}$  each user

### what if we don't have features

let's say we have  $w^j, b^j$  parameters,

so,  $w^j \cdot x^z + b^j = y^{(z,j)}$

eg:-  $w^1 x^1 + b^1 = 5$   
 $w^2 x^1 + b^2 = 5$

Since we have enough formulas for same  $x^1 = \begin{bmatrix} - \\ - \\ \vdots \end{bmatrix}$  can be calculate

So now to learn  $x^1, x^2, \dots, x^{n_m}$

$$J(x^1, x^2, \dots, x^{n_m}) = \frac{1}{2} \sum_{z=1}^{n_m} \sum_{j:r(z,j)=1} (w^j \cdot x^z + b^j - y^{(z,j)})^2 + \frac{\lambda}{2} \sum_{z=1}^{n_m} \sum_{k=1}^n (x_k^z)^2$$

min

here we learn for features not parameters

\*By combining both cost functions.  $\square$

$$\min_{w,b,x} J(w,b,x) = \frac{1}{2} \sum_{(i,j): r(i,j)=1} (w^j \cdot x^i + b^j - y^{(i,j)})^2 + \frac{\lambda}{2} \sum_{j=1}^{n_u} \sum_{k=1}^n (w_k^j)^2 + \frac{\lambda}{2} \sum_{i=1}^{n_m} \sum_{k=1}^n (x_k^i)^2$$

how to minimize,

Collaborative Filtering

Gradient descent,

repeat {

$$w_i^j = w_i^j - \alpha \frac{\partial J(w,b,x)}{\partial w_i^j}$$

$$b^j = b^j - \alpha \frac{\partial J(w,b,x)}{\partial b^j}$$

$$w_k^i = w_k^i - \alpha \frac{\partial J(w,b,x)}{\partial w_k^i}$$

← this is new rather before gradient descent

}

## Generate Algorithm for Binary labels

example:-

|          |                      |                         |       |
|----------|----------------------|-------------------------|-------|
| j ⇒ user | j <sup>th</sup> user | Purchase what Shown     | 1 / 0 |
|          | "                    | like an item            | 1 / 0 |
|          | "                    | Spend 30\$ with an item | 1 / 0 |
|          | "                    | Click on item           | 1 / 0 |

for binary labels, predict probability  $y^{(i,j)} = 1$

so. we use  $\underbrace{g(w^j \cdot x^i + b^j)}_z$  where  $g(z) = \frac{1}{1 + e^{-z}}$

so. cost,

now,  $y^{(i,j)} : f(w,b,x) = g(w^j \cdot x^i + b^j)$

loss for single example  $L(f(w,b,x), y^{(i,j)}) = -y^{(i,j)} \log(f(w,b,x)) - (1 - y^{(i,j)}) \log(1 - f(w,b,x))$

cost for  $(i,j): r(i,j)=1$

$$J(w,b,x) = \sum_{(i,j): r(i,j)=1} L(f(w,b,x), y^{(i,j)})$$

all examples.

# Mean normalization

| movie name | David | Eve | Ann | Eve ← new user |
|------------|-------|-----|-----|----------------|
|            | 5     | 2   | 0   | ?              |
|            | ?     | 1   | 1   | ?              |
|            | 0     | 1   | 3   | ?              |

Mean of each row

$$\begin{bmatrix} 5 & 4 & 0 & ? \\ ? & 1 & 1 & ? \\ 0 & 1 & 2 & ? \end{bmatrix} \Rightarrow \begin{bmatrix} 3 \\ 1 \\ 1 \end{bmatrix} = \mu$$

Subtract mean

$$\begin{bmatrix} 2 & 1 & -3 & ? \\ ? & 0 & 0 & ? \\ -1 & 0 & 1 & ? \end{bmatrix}$$

for user  $j$ , on movie  $i$

$$f(u, i, x) = w^j \cdot x^i + b + \mu_i$$

Can't be negative

\* normalize column if new movie add never rate by anyone.

## Finding related items

The features  $x^i$  of  $i$  are items.

To find items related to it, find item  $k$ ,  $x^k$  similar  $x^i$

For smallest distance.

$$\sum_{l=1}^n (x_l^k - x_l^i)^2$$

↓ vectorize

$$\|x^k - x^i\|^2$$

examples

| $l$      | $x^1$ | $x^2$ | $x^k$ | $x^i$ |
|----------|-------|-------|-------|-------|
| $l=1$    |       |       |       |       |
| $l=2$    |       |       |       |       |
| $\vdots$ |       |       |       |       |
| $l=n$    |       |       |       |       |

vector

## Content-based filters

ex:

User features →  $\begin{bmatrix} \vdots \\ x_u^{(j)} \\ \vdots \end{bmatrix}$  ←  $j^{th}$  user

vector

Movies features →  $\begin{bmatrix} \vdots \\ x_m^{(i)} \\ \vdots \end{bmatrix}$  ←  $i^{th}$  movie

vector

Predict ratings of  $j^{th}$  user on  $i^{th}$  movie,

$$V_u^{(j)} \cdot V_m^{(i)}$$

Calculate from  $x_u^j$

from  $x_m^i$

## Filtering

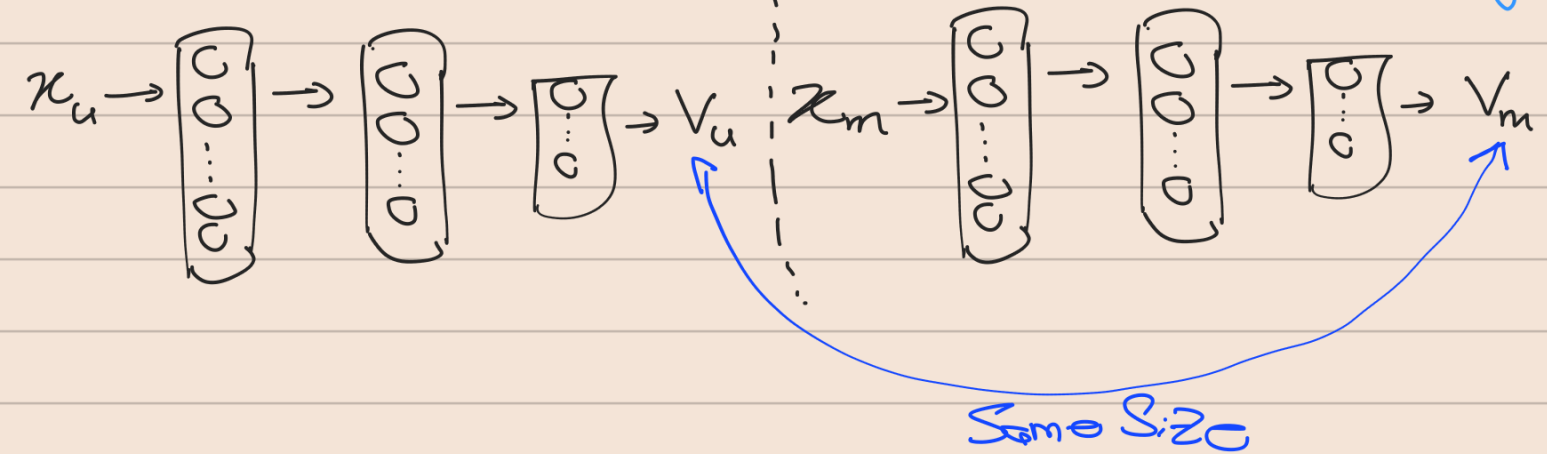
### Collaborative

Recommend based on ratings of users who has similar rating as you.

### Content based

Recommend using features of user and item find good match.

## Neural network architecture for Content based filtering.



Prediction:  $V_u^j \cdot V_m^i$

$g(V_u^j \cdot V_m^i)$  to predict probability  $y^{z,j} = 1$

\* Cost function,  

$$J = \sum_{(z,j): r(z,j)=1} (V_u^j \cdot V_m^i - y^{z,j})^2 + \text{NN regularization term,}$$
 ← neural network

\* To find similar movies  $z$ :-  

$$\| V_m^K - V_m^z \|^2 \text{ Small}$$
 ↑  
 vectors

## Reinforcement learning

eg:- robot walking

Places  
 $s \rightarrow \text{State} = \text{Current position}$   
 $a \rightarrow \text{action} = \text{action take from current position}$ 
 many action can take

$R_i \rightarrow$  rewards = the reward given ( $R_1, R_2, R_3 \dots$ )

$\gamma \rightarrow$  discount factor =  $\gamma$

$$\text{return} = R_1 + \gamma R_2 + \gamma^2 R_3 + \dots$$

$\uparrow$  reward in current position       $\uparrow$  next pos.

$$\text{Policy} = \pi(S)$$

### State action Value Function

$Q(S, a) =$  Returns if you • start in  $S$

• take  $a$  action (once)

• then behave optimally after that

eg:

$\gamma = 0.5$

$$Q(2, \rightarrow) = 0 + 0 \times 0.5 + 0 \times 0.5^2 + 100 \times 0.5^3$$
$$= 12.5$$

then back to 100.

$\pi$

this is optimal path after move to 3 position.

eg:  $Q(4, \leftarrow) = 0 + 0.5 \times 0 + 0.5^2 \times 0 + 0.5^3 \times 100$

$$= 12.5$$

### Picking action $a$ .

best return from  $S$  state is  $a$ , s.t  $\max Q(S, a)$

eg:

$S=4, \quad Q(4, \leftarrow) = 12.5 \quad Q(4, \rightarrow) = 10$

So, pick  $a = \leftarrow$ , in 4 position.

### Bellman Equation

$S$ : Current state       $R(S) =$  reward in  $S$

$S'$ : State after action  $a$

$a$ : Current action

$a'$ : action taken in  $S'$

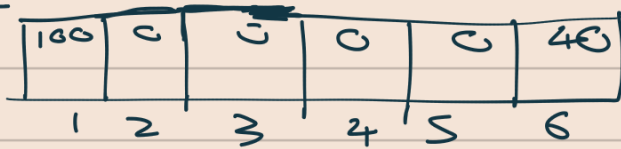
$$Q(S, a) = R_1 + \gamma R_2 + \gamma^2 R_3 + \dots \quad \text{optimal}$$

$$= R_1 + \gamma (R_2 + \gamma R_3 + \dots)$$

$$Q(S, a) = R(S) + \gamma \max_{a'} Q(S', a')$$

gives optimal from starting  $S'$

eg:-



$$Q(2, \rightarrow) = R(2) + 0.5 \max_{a'} Q(3, a')$$

$$= 0 + (0.5) \cdot 25$$

$$= 12.5$$

this is optimal path of  $Q(2, \rightarrow)$

maximum return from 3

Expected Return = Average  $(R_1 + \gamma R_2 + \gamma^2 R_3 + \dots)$

$$= E(R_1 + \gamma R_2 + \gamma^2 R_3 + \dots)$$

no command  $a = \rightarrow$  but robot go  $\leftarrow$   
so, there are many possibilities for return value for any state.

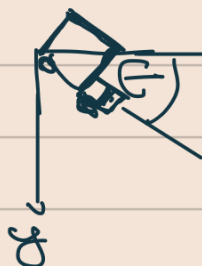
General Bellman

$$\therefore Q(S, a) = R(S) + \gamma E[\max_{a'} Q(S', a')]$$

\* In some case state  $S$  can be a vector.

eg:-

TRUCK moving



$$S = \begin{bmatrix} x \\ y \\ \theta \\ \omega \end{bmatrix}$$

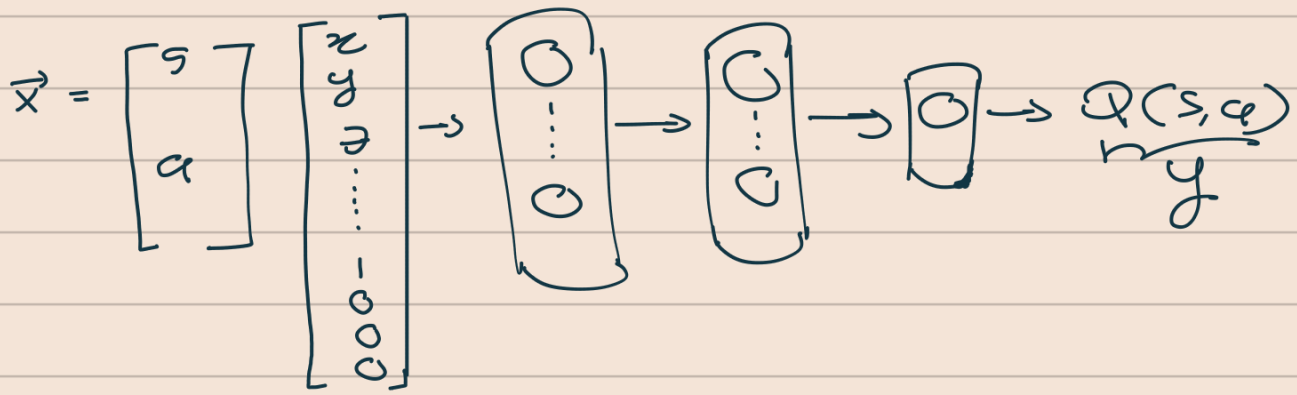
eg:-

helicopter

$$S = \begin{bmatrix} x \\ y \\ z \\ \phi \leftarrow \text{roll} \\ \theta \leftarrow \text{Pitch} \\ \psi \leftarrow \text{yaw} \\ \vdots \end{bmatrix}$$

# Deep Reinforcement Learning

eg:- Moon Rover Landing



Compute,

$Q(s, \text{left})$     $Q(s, \text{right})$     $Q(s, \text{main})$     $Q(s, \text{nothing})$   
 find maximum and do it.

## How to find parameters in NN

\* in training (or Simulators we find)

Current Position  $(s^{(1)}, a^{(1)}, R(s^{(1)}), s'^{(1)})$   
 do a action   get the reward   new position  
 $(s^{(2)}, a^{(2)}, R(s^{(2)}), s'^{(2)})$   
 to compute  $x$    To compute  $y$

$$Q(s, a) = R(s) + \gamma \max_{a'} Q(s', a')$$

we get  $x$  as this

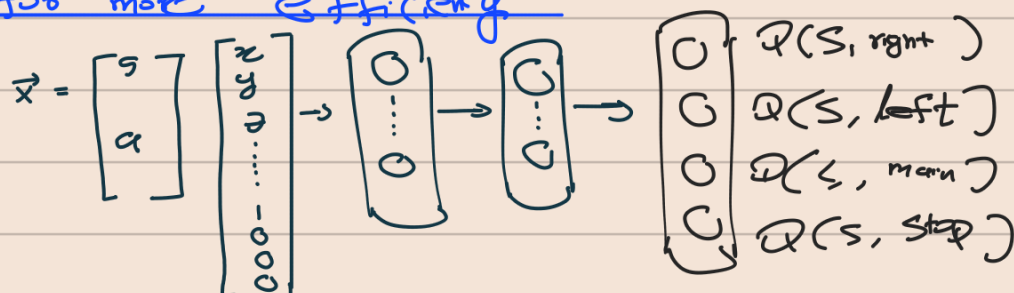
$$y^{(1)} = R(s^{(1)}) + \gamma \max_{a'} Q(s'^{(1)}, a')$$

$f_{w,b}(x)$

$x^{(1)} = (s^{(1)}, a^{(1)})$

Train s.t  $Q_{\text{new}}(s, a) \approx y$  (find  $w, b$ ) Parameters.

## for more efficiency



well done!

